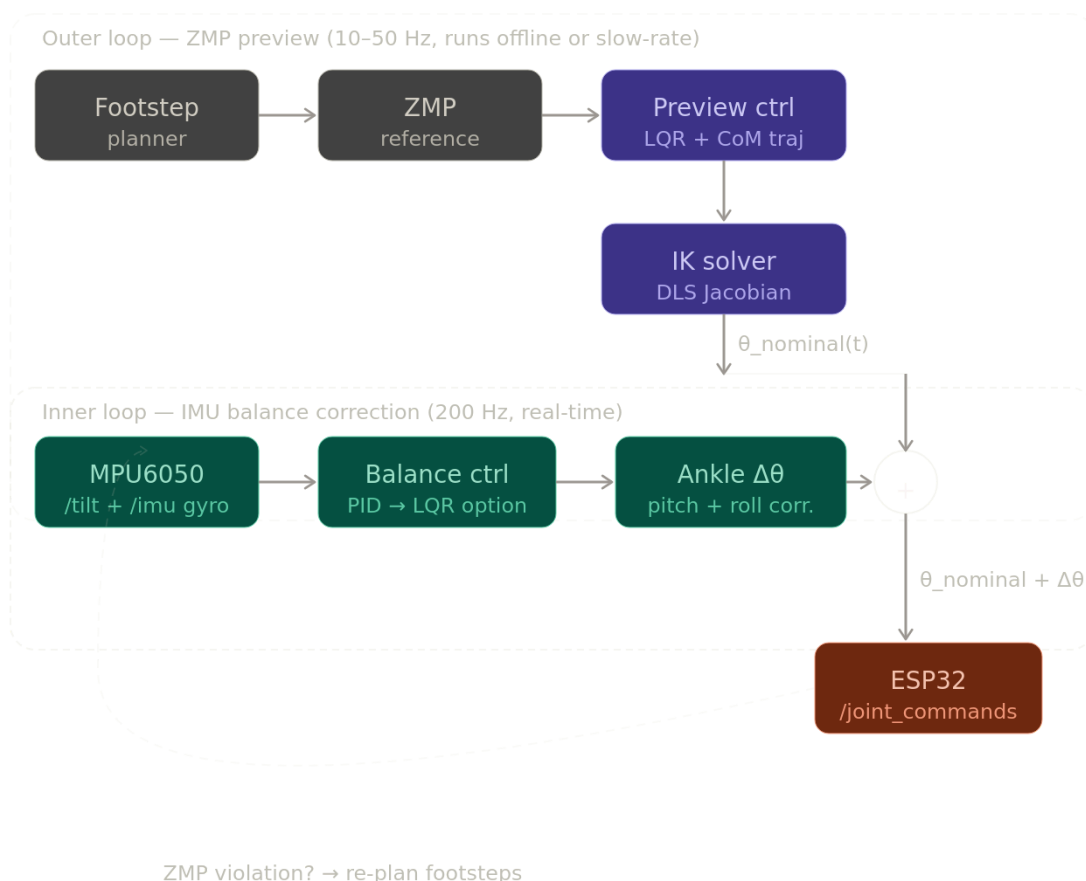


Outer loop vs inner loop

The reason is architectural. Your balance node does something fundamentally different from the preview controller. The preview controller is a *planner* — it generates a geometrically stable CoM trajectory assuming the robot will execute it perfectly on flat ground. The MPU6050 balance controller is a *disturbance rejector* — it corrects for what the planner couldn't predict: ground slope, servo lag, impact shocks, uneven terrain. These two responsibilities belong in separate loops running at different rates (preview: 10–50 Hz; IMU correction: 200 Hz). Collapsing them into one LQR would mean either running an expensive trajectory optimizer at 200 Hz, or running IMU correction at 50 Hz — both bad tradeoffs on an ESP32-class system.

What to change: upgrade the inner loop from PID to LQR for better disturbance rejection, and add a ZMP feedback path that re-plans footsteps when the IMU detects a fall the planner didn't anticipate.



OFFLINE (planning, avant exécution)

footstep_planner.py

"où vont aller les pieds dans les 8 prochains pas"



zmp_reference.py

"quelle trajectoire ZMP pour rester stable"



preview_controller.py (LQR)

"quelle trajectoire CoM pour suivre ce ZMP"

il REGARDE EN AVANT sur 1.5s de futur



ik_solver.py (géométrie simple)

"quels angles joints pour chaque point de la trajectoire"

→ produit joint_traj[N×18] — tout calculé d'avance



ZMP verification

"est-ce que cette trajectoire est stable ?"

si oui → on execute

ONLINE (exécution, temps réel)

pipeline publie /joint_nominal à 200Hz

(rejoue le tableau pré-calculé step by step)



balance_controller.py

lit l'IMU → détecte si le robot penche

ajoute $\Delta\theta_{\text{ankle}}$ pour corriger



ik_vectors_DLS (Wampler DLS) ← TON VRAI IK

reçoit /com_trajectory ← la position CoM courante
recalcule les angles en temps réel
corrige les erreurs du planning offline
gère les singularités avec DLS
publie /joint_commands à 100Hz

↓
joint_bridge
→ /leg_controller/commands (radians)
↓
servos bougent

DOOONNNCCC

OUTER LOOP — Preview Control (offline / slow rate 10-50Hz)

—
LQR avec horizon futur (N=300 steps = 1.5s)

Résout : $\min J = \sum (e_zmp^2 + r \cdot u^2)$
sur les N prochains pas

→ calcule trajectoire CoM optimale
→ garantit que ZMP reste dans le polygone de support
→ produit joint_traj[N×18] via ik_solver.py
→ publie /joint_nominal

Gain K calculé UNE FOIS offline via Riccati :
 $P = \text{solve_discrete_are}(A, B, Q, R)$
 $K = (R + B^T P B)^{-1} B^T P A$

INNER LOOP — Balance Control (online, 200Hz)

—
LQR/PID avec feedback IMU temps réel

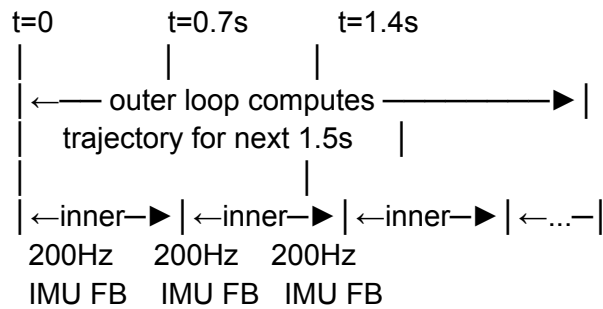
État $x = [\text{tilt_pitch}, \text{tilt_roll}, \omega_pitch, \omega_roll]$
Erreur = tilt mesuré par MPU6050

→ calcule $\Delta\theta_ankle$ pour corriger le déséquilibre
→ ajoute à /joint_nominal
→ publie /joint_commands corrigés

C'est un régulateur local — réagit aux perturbations
que le planning offline n'a pas prévu :

- poussées externes
- sol irrégulier
- erreurs de modèle

TIMELINE d'un pas :



outer replans every $\sim 0.3s$

inner corrects every 5ms

Résumé des deux LQR :

	Outer (Preview)	Inner (Balance)
Objectif	minimise ZMP error	minimise tilt error
Horizon	1.5s futur	temps réel
Rate	10-50 Hz	200 Hz
Input	footstep plan	IMU measurements
Output	/joint_nominal	$\Delta\theta_{\text{ankle}}$
Gain K	calculé offline	calculé offline
	rejoué online	rejoué online

FULL ARCHITECTURE

